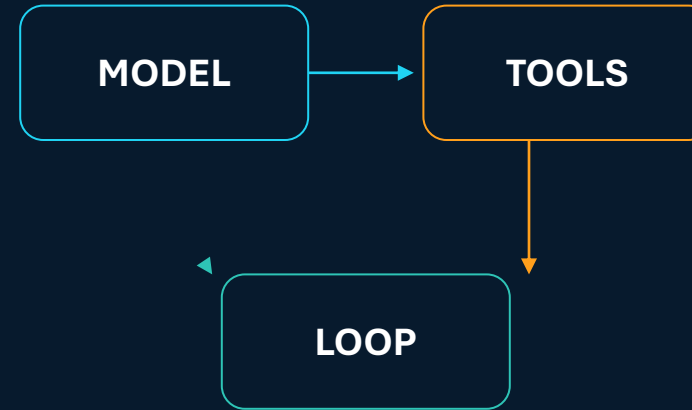


AI Agents

Modulo 1 — Artificial Intelligence (laboratorio)



Obiettivo: progettare software che osserva, decide e agisce

Dove siamo arrivati

Dal prompt engineering ai sistemi agentici



Pipeline già nota agli studenti

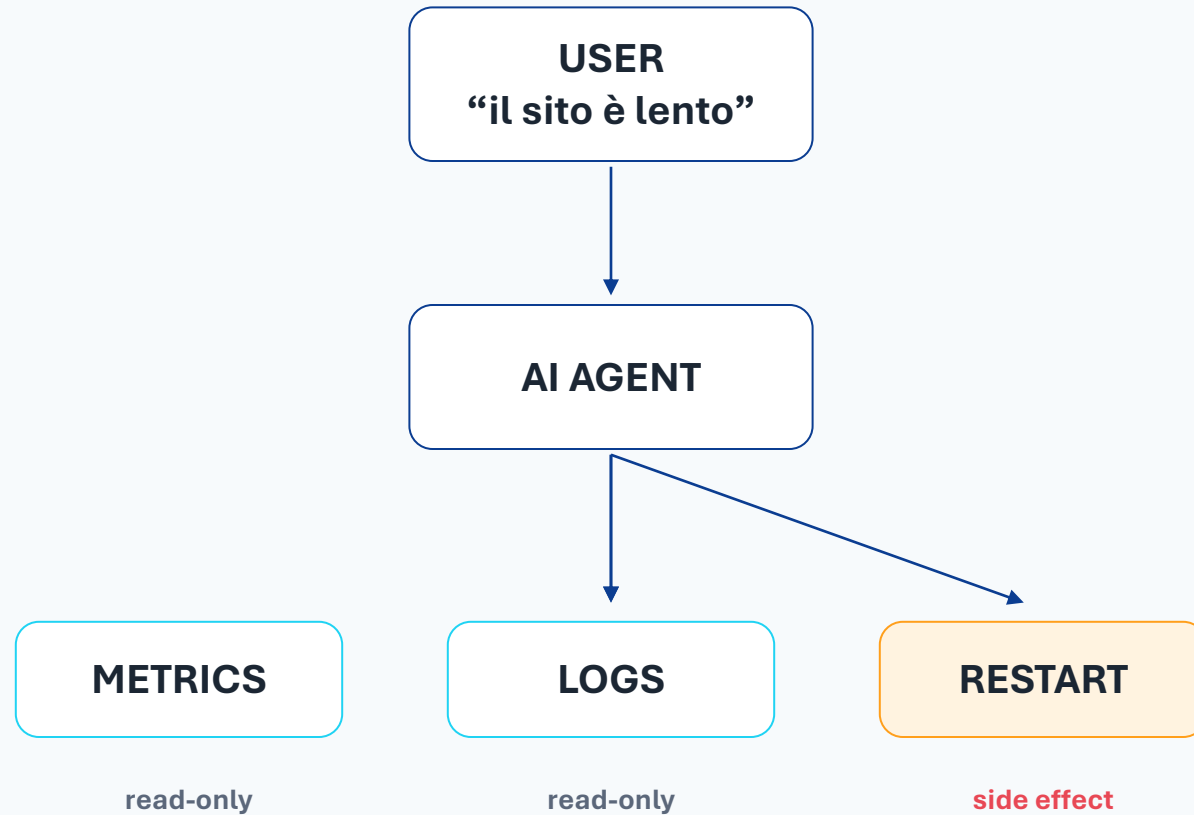


Nuovo paradigma: il sistema decide quali strumenti usare

Caso di studio

Autonomous IT Operations Agent

- Un portale aziendale è lento
- L'agente dispone di metriche, log e stato servizi
- Alcuni strumenti leggono, altri modificano lo stato
- Le azioni rischiose richiedono approvazione



Risultati di apprendimento

Cosa deve saper fare lo studente al termine

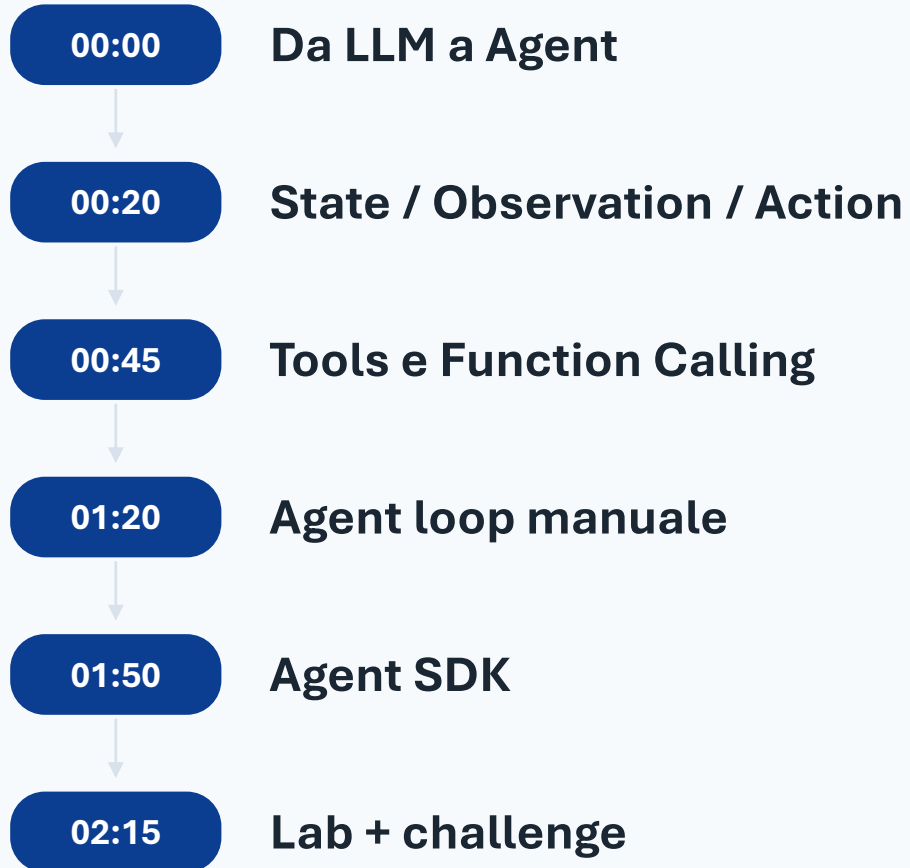
- Distinguere LLM, chatbot, workflow e agent
- Comprendere state, observation, action e environment
- Implementare un agent loop manuale
- Usare tool/function calling e JSON schema
- Gestire errori, max turns e trace
- Controllare side effect con human-in-the-loop

Agent = Model + Tools + State + ControlLoop

Il focus non è il framework: è l'architettura.

Scaletta delle 3 ore

Teoria breve, coding guidato, laboratorio e challenge



OUTPUT
notebook + sistema
funzionante

Un LLM non è un agente

Una chiamata semplice è una trasformazione input → output



$$y = f(x)$$

- Il modello genera una risposta
- Non conosce lo stato reale dei nostri sistemi
- Non esegue azioni da solo

Chatbot vs Workflow vs Agent

Tre strutture concettuali diverse

CHATBOT
question → answer

```
response = llm(prompt)
```

WORKFLOW
step 1 → step 2 → step 3

```
A()  
B()  
C()
```

AGENT
decide il percorso

```
while not done:  
    action = decide()  
    observe(action)
```

La differenza chiave è chi decide la prossima azione.

Anatomia di un Agent

Componenti minimi e componenti professionali

MODEL

TOOLS

STATE

LOOP

Agent = Model + Tools + State + ControlLoop

Nei sistemi reali si aggiungono:

- memory,
- guardrails,
- permissions,
- observability

Environment

Il mondo con cui l'agente interagisce



L'agente non “sa” tutto: osserva parti dell'ambiente tramite strumenti.

State

Lo stato del sistema al tempo t

```
s_t = {  
  server: "web-01",  
  cpu: 94,  
  memory: 78,  
  service: "running"  
}
```

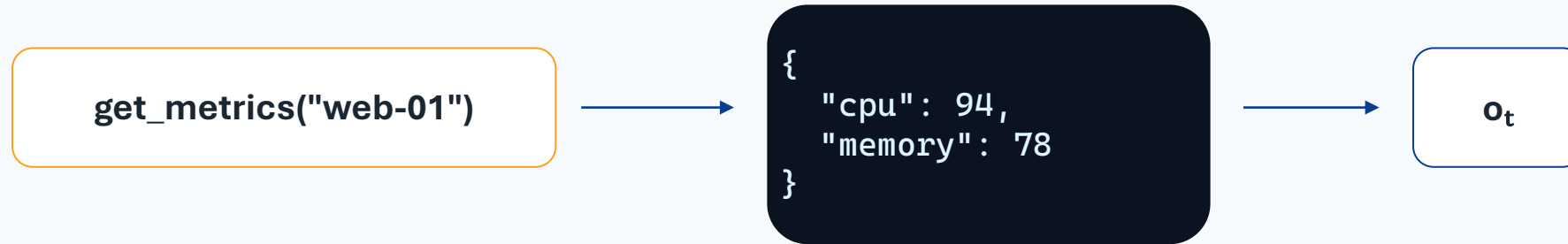


S_t

- Lo stato reale può essere più grande di ciò che osserviamo
- Un tool produce una vista parziale
- Le decisioni dipendono dalle osservazioni raccolte

Observation

Informazione ottenuta dall'ambiente



Un'osservazione è il risultato leggibile di un'azione informativa.

Action

L'azione scelta dall'agente

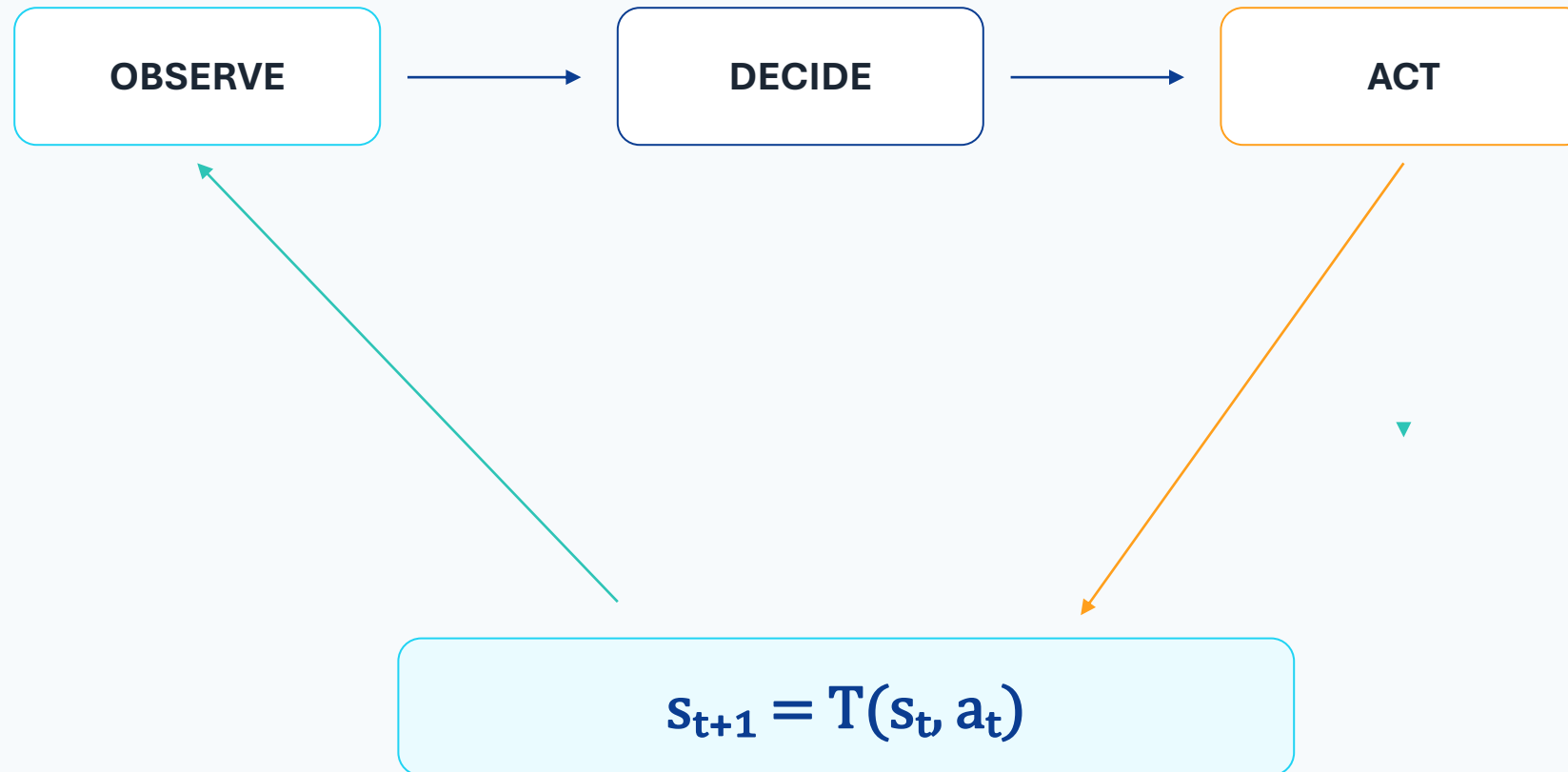
- **Read action:** leggere metriche, log, stato
- **Write action:** riavviare, inviare, modificare
- **Delete/pay/send/execute:** azioni ad alto rischio

$$a_t \sim \pi(a \mid h_t)$$

La formula dice: data la storia h_t , il sistema seleziona una possibile azione a_t .

Observe → Decide → Act

Il loop agentico



Agent Loop in pseudocode

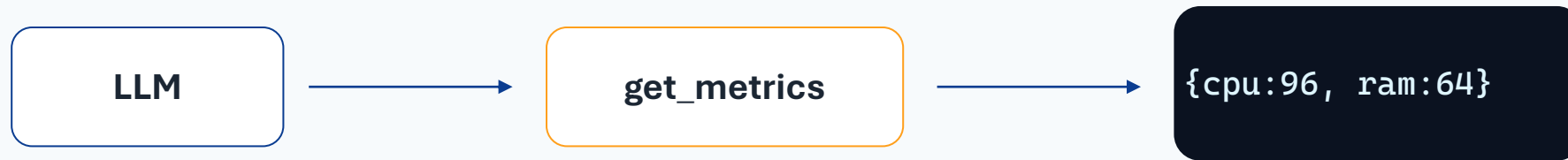
Il pezzo di codice più importante della lezione

```
while not finished:
    response = model(state, tools)
    if response contains tool_call:
        result = execute_tool()
        state = update_state(result)
    else:
        finished = True
```

- Il modello può chiedere un tool
- Python esegue davvero la funzione
- Il risultato torna nel contesto
- Il ciclo continua fino alla risposta finale

Perché servono i Tools?

Il modello non conosce lo stato live dei sistemi



I tool collegano il ragionamento del modello al mondo operativo.

Tool Calling

Il modello genera una richiesta strutturata

```
{  
  "name": "get_metrics",  
  "arguments": {  
    "server": "web-01"  
  }  
}
```



APPLICATION
valida + esegue



PYTHON
function

Il modello non esegue direttamente Python: lo chiede all'applicazione.

Tool Schema

Il contratto software tra agente e applicazione

Tool = Name + Description + InputSchema

```
{  
  "name": "get_metrics",  
  "description": "Returns CPU and RAM utilization",  
  "parameters": {  
    "server": {"type": "string"}  
  }  
}
```

**Una descrizione
vaga produce
scelte peggiori.**

Tool descriptions

La descrizione guida la selezione dello strumento

get_data
“Gets data”

get_server_metrics
“Use when diagnosing performance issues”

La differenza non è estetica: cambia il comportamento dell’agente.

Livelli di autonomia

Non sempre più autonomia significa migliore sistema



Autonomy ↑ ⇒ Risk ↑

Read vs Write

Side effect e controllo umano

READ
get_metrics
read_log

basso rischio

WRITE
restart_service
create_ticket

approval

CRITICAL
delete / pay / send

approval forte

```
def restart_service(..., approved=False):  
    if not approved:  
        return {"requires_approval": True}
```

Human-in-the-loop

Le azioni con impatto richiedono conferma



Pattern: diagnosis → propose action → approval → execute → verify

Loop infiniti

Ogni agente deve avere limiti operativi

```
get_metrics  
↓  
read_log  
↓  
get_metrics  
↓  
read_log  
↓  
...
```

$$N_{\text{turns}} \leq N_{\text{max}}$$

```
MAX_TURNS = 8
```

- limita costo
- limita latenza
- evita cicli patologici
- semplifica debugging

Error handling

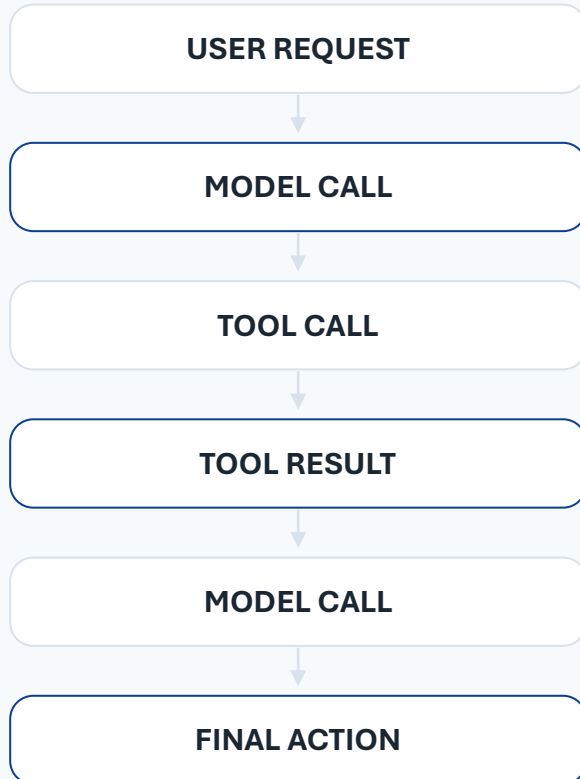
Un fallimento può diventare una nuova osservazione



```
{  
  "success": false,  
  "error": "SERVER_UNREACHABLE"  
}
```

Observability

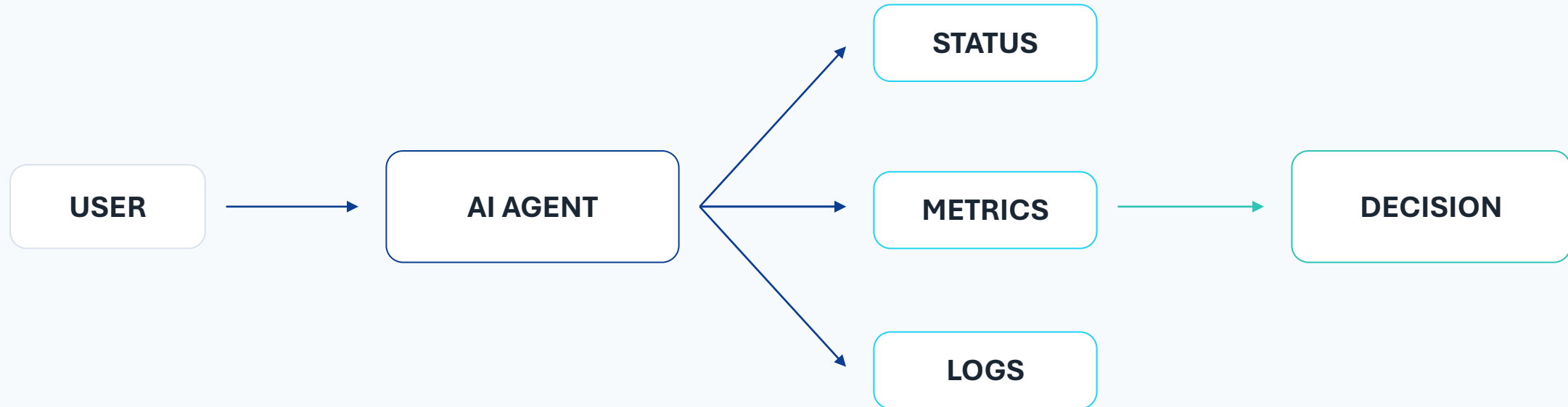
Perché l'agente ha preso quella decisione?



Trace = sequenza degli eventi necessari per debugging, audit e miglioramento.

Il nostro sistema

Architettura del laboratorio



Notebook – cosa troveremo

Sequenza tecnica delle celle principali

- Ambiente simulato
- Tools read-only
- Tool con side effect
- Tool registry
- Planner simulato
- Agent loop manuale
- Trace e human approval
- Demo opzionale API/SDK

```
servers = {...}  
TOOL_REGISTRY = {...}  
state = run_manual_agent(...)  
print_trace(state)
```

Manual Agent Loop

Versione didattica senza nascondere il meccanismo

```
for turn in range(MAX_TURNS):  
    decision = planner(state)  
  
    if final_answer(decision):  
        break  
  
    result = execute_tool(decision)  
    state.observations.append(result)
```

**Prima si capisce il ciclo.
Solo dopo si usa un SDK.**

Agent SDK

L'astrazione diventa utile dopo aver capito il loop

```
agent = Agent(  
    name="IT Operations Agent",  
    instructions="Diagnose problems",  
    tools=[server_metrics, server_logs]  
)  
  
result = Runner.run_sync(agent, task)
```

**SDK gestisce
turni + tools + stato**

**La domanda progettuale diventa:
quanto controllo voglio delegare al
framework?**

Laboratorio – cosa faremo

Aggiungere diagnostica sulle connessioni DB

- Completare `get_database_connections()`
- Registrare il nuovo tool
- Usarlo nel planner
- Raccogliere evidenze prima della conclusione
- Non riavviare servizi senza approval

```
{  
  "active": 99,  
  "max": 100,  
  "utilization": 0.99  
}
```

$\text{utilization} = \text{active} / \text{max}$

Challenge finale

The Slow Website Incident

web-01
CPU normale
log normali

api-01
DB waiting
request timeout

db-01
99 / 100 connessioni

Obiettivo: formulare una causa probabile, non una certezza assoluta.

$$99 / 100 = 99\%$$

Quando NON usare un Agent

Un agente non è sempre la soluzione giusta

WORKFLOW
sequenza nota

AGENT
decisione dinamica

Usa un workflow se la sequenza è stabile e deterministica.

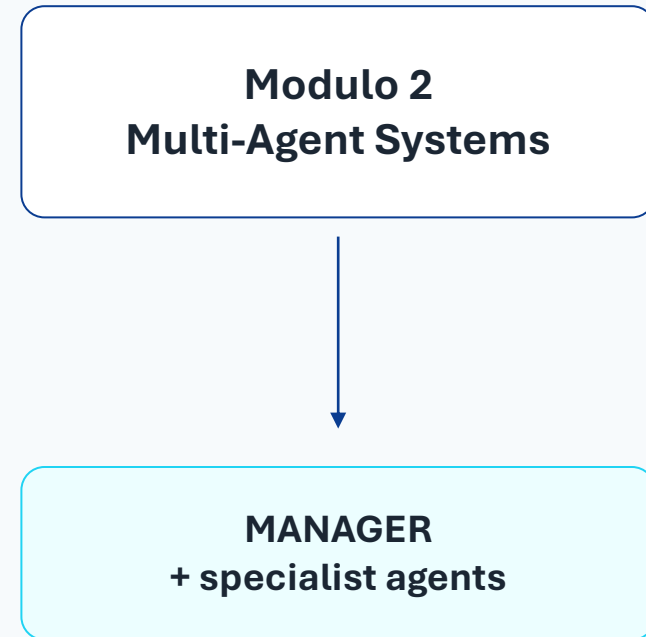
Usa un agent se non conosci a priori quali strumenti serviranno.

Costo, latenza e rischio aumentano con l'autonomia.

Takeaways

Cinque concetti da portare a casa

- Agent $\neq$ LLM
- Agent = Model + Tools + Loop
- Action $\rightarrow$ Observation $\rightarrow$ New Decision
- Autonomy $\uparrow$ implica Risk $\uparrow$
- Usare agenti solo quando serve decisione dinamica



Riferimenti tecnici

Documentazione utile

- OpenAI Agents SDK: Agent, Runner, tools, guardrails, tracing
- OpenAI Function Calling / Responses API
- JSON Schema per interfacce strutturate
- Pattern: human-in-the-loop e approval su side effect

Fonti operative usate per allineare gli esempi: docs OpenAI Agents SDK e Function Calling.